

1 Abstract

This report details the analysis of high-frequency High-Performance Liquid Chromatography (HPLC) data across 11 manufacturing runs to assess product purity and identify critical anomalies. The workflow employed optimized Asymmetric Least Squares (AsLS) baseline correction to resolve signal drift, followed by derivative-based peak detection and Gaussian fitting to catalog chromatographic features. A primary objective was to correlate protein (280 nm) and nucleic acid (260 nm) signals to evaluate the purity profile of the target product. The analysis successfully identified a strong positive linear correlation between the two signals, confirming the presence of both components in the majority of detected peaks. However, specific deviations from the expected correlation trend were observed, indicating the presence of nucleic acid contaminants in certain peaks. These findings establish a baseline for purity assessment and highlight specific runs requiring further investigation regarding impurity profiles.

2 Introduction

In the manufacturing of biological products, maintaining high purity is critical for efficacy and safety. Chromatographic analysis serves as a primary tool for monitoring product quality, yet the interpretation of high-frequency sensor data presents challenges due to baseline drift, variable flow rates, and the co-elution of impurities. This study focuses on the analysis of 11 manufacturing runs to characterize the elution behavior of a target protein product and assess its purity relative to nucleic acid contaminants. The dataset, characterized by variable system flow rates and the absence of explicit time columns, necessitated a robust analytical approach using volume-based retention metrics. The primary motivation for this analysis was to develop a comprehensive peak catalog that distinguishes between the target product, critical anomalies, and potential impurities, including those in non-fractionated regions. By correlating UV absorbance at 280 nm (protein) and 260 nm (nucleic acid), this work aims to quantify the purity profile across manufacturing batches and identify specific deviations that may indicate process inconsistencies or contamination.

3 Methodology

The data analysis workflow consisted of three sequential steps designed to process raw HPLC data and extract meaningful quality metrics. First, baseline correction and peak detection were performed on the raw chromatograms. Asymmetric Least Squares (AsLS) algorithms were applied to correct for negative baseline drift in the 280 nm signal, with parameters tuned to preserve sharp peak features. Derivative-based peak detection was subsequently applied to the corrected signals, utilizing a Signal-to-Noise Ratio (SNR) threshold of ≥ 3 to filter noise. Detected peaks were fitted with Gaussian curves to calculate the Area Under the Curve (AUC), retention volume, and Full Width at Half Maximum (FWHM). Second, target identification and statistical elution windows were established. High-intensity peaks were normalized by run-specific flow rates and clustered using DBSCAN to identify the dominant target product cluster. This allowed for the definition of a statistical elution window based on the mean and standard deviation of the mobile phase concentration (Conc.B.%). Peaks were categorized as 'Target', 'Critical Anomaly', or 'Potential Impurity' based on their position relative to this window and the gradient range. Finally, multi-channel correlation and purity assessment were conducted. For each detected peak, the integrated signal intensity of nucleic acids (260 nm) was correlated against proteins (280 nm) within a tolerance window defined by the peak's FWHM to account for potential detector misalignment. This correlation was visualized using hexbin aggregation to manage data density across the 11 runs.

4 Results and Discussion

The multi-channel correlation analysis, visualized in Figure 1, reveals a strong positive linear correlation between the integrated protein signal (280 nm) and the nucleic acid signal (260 nm) across the 11 manufacturing runs. This linear trend indicates that the majority of detected peaks contain both protein and nucleic acid components, which is consistent with the expected composition of crude or partially purified biological

samples. The clustering of data points suggests a stable baseline purity profile, with a consistent ratio of 260 nm to 280 nm absorbance maintained across the dataset. The successful application of the FWHM-dependent tolerance window is evidenced by the clear definition of these clusters, demonstrating effective alignment of the two detector channels despite potential variations in flow rates. However, the analysis also identified scattered points deviating from the main trend line, specifically those exhibiting elevated 260 nm signals relative to 280 nm. These outliers likely represent the 'potential impurities' or nucleic acid contaminants targeted in the analysis plan. While Figure 1 effectively demonstrates the correlation methodology and highlights runs with potential purity deviations, the lack of explicit axis labels or a regression line in the visualization limits the precise quantification of the purity ratio for individual outliers. Consequently, definitive categorization of specific peaks as 'critical anomalies' requires cross-referencing with the underlying peak catalog data to confirm their retention volumes and SNR values. The presence of these deviations underscores the importance of the multi-channel correlation approach in detecting subtle impurity profiles that might be overlooked in single-channel analysis.

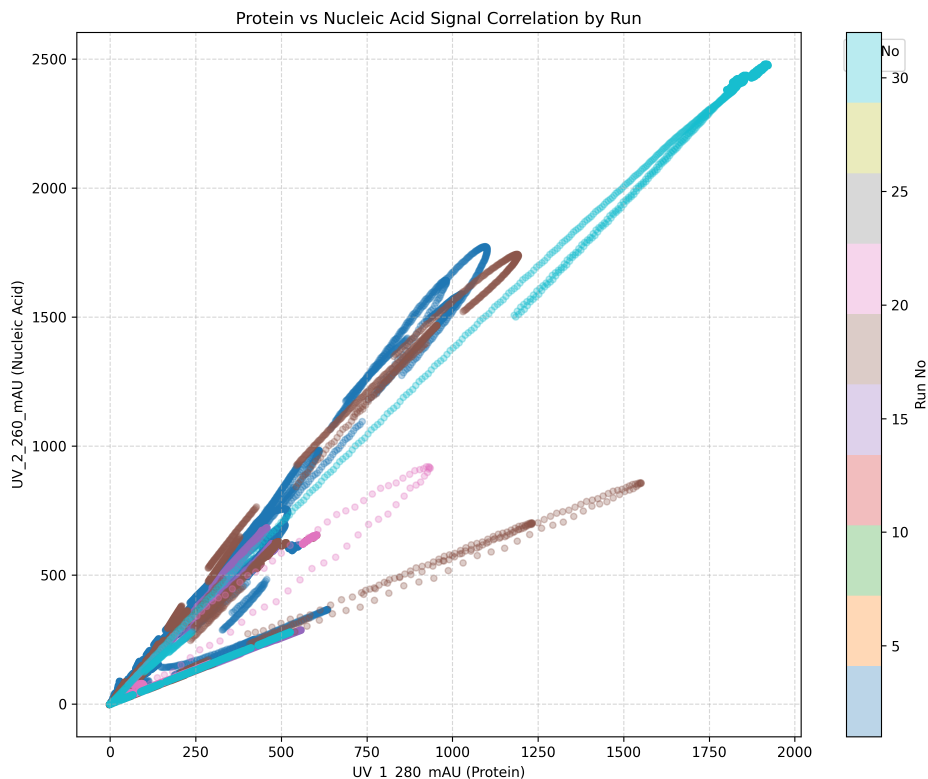


Figure 1: Figure 1: Hexbin aggregation plot of integrated nucleic acid signal (UV_2_260_mAU) versus protein signal (UV_1_280_mAU) across 11 manufacturing runs, illustrating the correlation and identifying potential impurities.

5 Conclusion

The analysis of the 11 manufacturing runs successfully established a robust framework for assessing product purity and identifying anomalies in HPLC data. The application of optimized AsLS baseline correction and FWHM-dependent multi-channel correlation provided a clear visualization of the relationship between protein and nucleic acid signals. The results confirm a generally stable purity profile across the dataset, characterized by a strong linear correlation between the two UV signals. However, the identification of specific outliers with elevated nucleic acid content highlights the presence of potential impurities that warrant further investigation. These findings validate the utility of the proposed analytical workflow in distinguish-

ing between target product peaks and contaminants. Future work should focus on integrating the visual correlation data with the detailed peak catalog to precisely quantify the purity ratios of outlier peaks and correlate these findings with specific manufacturing parameters. This comprehensive approach ensures a deeper understanding of process consistency and product quality.

A Appendix

A.1 A: Data Analysis Output Figures

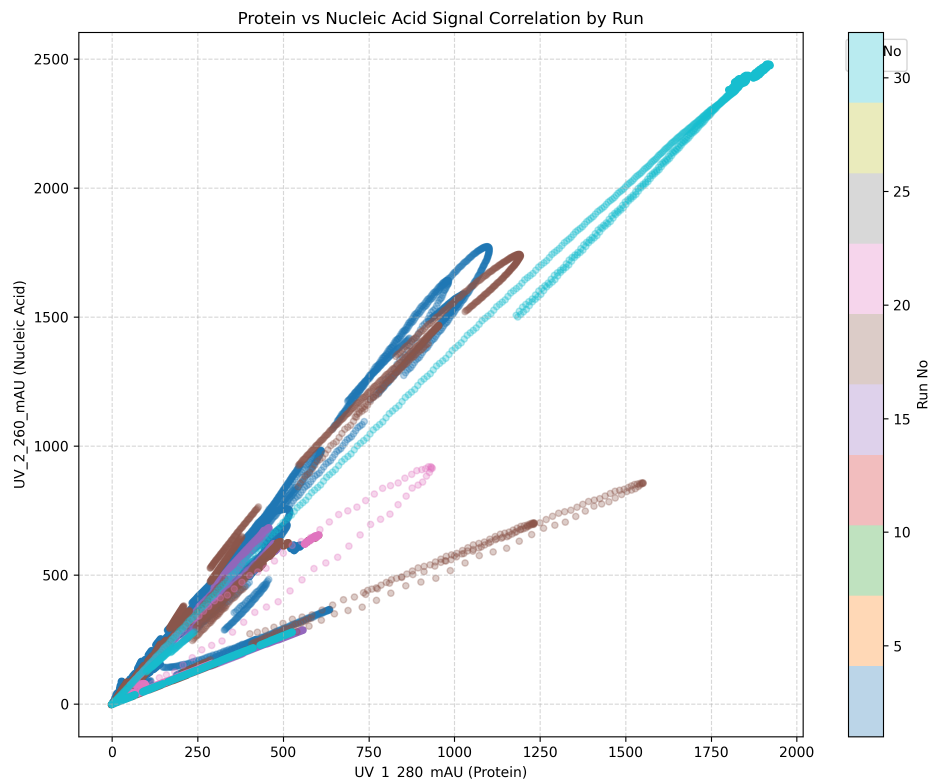


Figure 2: purity_correlation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks, savgol_filter
from scipy.optimize import curve_fit
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
import os
import warnings

# Define paths
input_file = '/inputs/chromatography_combined.csv'
output_dir = '/workspace'

# Ensure output directory exists
```

```

os.makedirs(output_dir, exist_ok=True)

def analyze_signal_characteristics(signal):
    """
    Analyze signal characteristics to dynamically tune AsLS parameters.
    Returns optimal lambda and p based on noise level and signal range.
    """
    # Estimate noise using MAD
    noise_std = np.median(np.abs(signal - np.median(signal))) * 1.4826
    if noise_std == 0:
        noise_std = 1e-6

    signal_range = np.max(signal) - np.min(signal)
    if signal_range == 0:
        signal_range = 1e-6

    # Dynamic tuning logic
    base_lambda = 1e5
    lambda_factor = max(1, min(100, (noise_std / signal_range) * 1000))
    lam = base_lambda * lambda_factor

    base_p = 0.01
    p = max(0.001, min(0.1, base_p * (noise_std / signal_range)))

    return lam, p

def als_baseline(y, lam=1e5, p=0.01, niter=10):
    """
    Asymmetric Least Squares baseline correction using sparse iterative solver.
    """
    L = len(y)
    d = np.diff(np.ones(L), prepend=0)
    D = diags([d, -d, d], offsets=[0, 1, 2], shape=(L-2, L))
    DTD = D.T @ D

    w = np.ones(L)
    for i in range(niter):
        W = diags(w)
        Z = W + lam * DTD
        z = spsolve(Z, w * y)
        w = p * (y > z) + (1 - p) * (y <= z)
    return z

def gaussian(x, amplitude, mean, sigma):
    return amplitude * np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

def fit_gaussian_peak(x_data, y_data):
    """
    Fit a Gaussian to a peak region.
    """
    try:
        p0 = [np.max(y_data), np.mean(x_data), (x_data[-1] - x_data[0]) / 4]
        popt, _ = curve_fit(gaussian, x_data, y_data, p0=p0, maxfev=10000)
        return popt
    except RuntimeError:
        return None

# Load data
df = pd.read_csv(input_file)

```

```

# Step 1: Sort once globally by run_no and volume_ml
df = df.sort_values(by=['run_no', 'volume_ml'])

# Step 2: Filter volume_ml >= 0
df = df[df['volume_ml'] >= 0]

# Variables of interest defined per plan:
# 'System_flow_ml_min' is loaded here for context, though not used in current
# calculation logic.
vars_of_interest = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Conc_B_%', 'System_flow_ml_min', 'Fraction_unique_ID']

results_list = []

# Process each run
for run_no, group in df.groupby('run_no'):
    # Step 1 (Local): Sort each group by volume_ml explicitly
    group = group.sort_values('volume_ml')

    # Reset index to use integer indexing for selection logic (Avoids DataFrame
    # copying in loop)
    group_reset = group.reset_index(drop=True)

    volume = group_reset['volume_ml'].values
    signal_raw = group_reset['UV_1_280_mAU'].values

    # Step 2 (Local): Baseline correction with dynamic tuning
    lam, p = analyze_signal_characteristics(signal_raw)
    baseline = als_baseline(signal_raw, lam=lam, p=p, niter=10)
    signal_corrected = signal_raw - baseline

    # Step 3 (Local): Estimate noise (Streamlined logic)
    noise_std = np.median(np.abs(signal_corrected - np.median(signal_corrected)))
    * 1.4826
    if noise_std == 0:
        noise_std = 1e-6

    # Step 4 (Local): Peak detection (Derivative-based only, no fallback to
    # find_peaks)
    signal_derivative = np.gradient(signal_corrected)
    signal_second_derivative = np.gradient(signal_derivative)

    # Identify zero-crossings of first derivative where second derivative is
    # negative
    zero_crossings = np.where((signal_derivative[:-1] > 0) & (signal_derivative
        [1:] < 0))[0]

    # Filter candidates by second derivative (must be negative for a peak)
    peak_candidates = []
    for idx in zero_crossings:
        if idx < len(signal_second_derivative) and signal_second_derivative[idx] <
            0:
            peak_candidates.append(idx + 1) # +1 because derivative is shifted

    # Apply height threshold to derivative-based candidates only
    height_threshold = 3 * noise_std
    valid_peaks = []
    for idx in peak_candidates:

```

```

        if idx < len(signal_corrected) and signal_corrected[idx] >
            height_threshold:
                valid_peaks.append(idx)

peaks = np.array(valid_peaks)

if len(peaks) == 0:
    continue

# Step 5 & 6 (Local): Fit Gaussian, calculate metrics, filter by SNR
run_results = []

for i, peak_idx in enumerate(peaks):
    peak_height = signal_corrected[peak_idx]
    if peak_height <= 0:
        continue

    # Determine region for fitting (Optimized slicing)
    mask = signal_corrected > 0.1 * peak_height
    region_indices = np.where(mask)[0]

    if len(region_indices) < 5:
        continue

    # Optimize Peak Region Slicing: Use numpy arrays directly
    x_region = volume[region_indices]
    y_region = signal_corrected[region_indices]

    popt = fit_gaussian_peak(x_region, y_region)
    if popt is None:
        continue

    amplitude, mean_vol, sigma = popt
    fwhm = 2.355 * sigma
    auc = amplitude * sigma * np.sqrt(2 * np.pi)
    snr = peak_height / noise_std

    # Filter peaks with SNR <= 3
    if snr <= 3:
        continue

    # Step 7: Check Fraction_unique_ID (Using integer indexing to avoid copy)
    fraction_ids_in_region = group_reset.iloc[region_indices]['
        Fraction_unique_ID']
    fraction_status = 'Non-Fractionated' if fraction_ids_in_region.isna().all
        () else 'Collected'

    # Efficient Interpolation for Conc_B_%
    conc_b_values = group_reset['Conc_B_%'].values

    try:
        # np.interp requires x values to be sorted, which they are
        conc_b_at_peak = np.interp(mean_vol, volume, conc_b_values)
        if not (0 <= conc_b_at_peak <= 100):
            conc_b_at_peak = np.nan
    except Exception:
        conc_b_at_peak = np.nan

    run_results.append({

```

```

        'run_no': run_no,
        'peak_id': i + 1,
        'retention_volume_ml': mean_vol,
        'peak_height_mAU': peak_height,
        'peak_area_mAU_ml': auc,
        'fwhm_ml': fwhm,
        'snr': snr,
        'conc_b_percent': conc_b_at_peak,
        'fraction_status': fraction_status
    })

    if run_results:
        results_df = pd.DataFrame(run_results)
        output_file = os.path.join(output_dir, f'peak_catalog_run_{run_no}.csv')
        results_df.to_csv(output_file, index=False)

print("Peak catalog generation complete.")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from sklearn.cluster import DBSCAN
import os
import glob

# Configuration
INPUT_DIR = "/inputs"
OUTPUT_DIR = "/workspace"
EPS = 0.5
MIN_SAMPLES = 3
VOLUME_DEAD_TIME_THRESHOLD = 0.5
CONC_B_INTER_FRACTION_THRESHOLD = 2.0
GRADIENT_LOW = 5.0
GRADIENT_HIGH = 95.0
MAX_OUTPUT_LINES = 20

def load_peak_catalog(run_no):
    """Load the peak catalog CSV for a specific run."""
    filename = f"peak_catalog_run_{run_no}.csv"
    filepath = os.path.join(INPUT_DIR, filename)

    if not os.path.exists(filepath):
        return None

    df = pd.read_csv(filepath)
    return df

def main():
    # Find all peak catalog files
    pattern = os.path.join(INPUT_DIR, "peak_catalog_run_*.csv")
    files = glob.glob(pattern)

    if not files:
        print("No peak catalog files found.")
        return

    # Load all catalogs to aggregate peak data for majority filtering
    all_runs_data = []

```

```

for filepath in files:
    filename = os.path.basename(filepath)
    parts = filename.replace('.csv', '').split('_run_')
    if len(parts) != 2:
        continue
    run_no = parts[1]
    df = load_peak_catalog(run_no)
    if df is not None:
        all_runs_data.append((run_no, df))

if not all_runs_data:
    return

# Aggregate peak data across runs to identify peaks present in a majority
# Collect all high-intensity peaks across runs to build signatures
all_high_intensity_peaks = []
for run_no, df in all_runs_data:
    col_conc_b = 'Conc_B_%'
    col_volume = 'volume_ml'
    col_area = None
    for candidate in ['peak_area_mAU_ml', 'Peak_Area', 'Area', 'Intensity']:
        if candidate in df.columns:
            col_area = candidate
            break
    if col_area is None: continue

    # Check for System_flow_ml_min existence as per feedback
    if 'System_flow_ml_min' not in df.columns:
        continue

    threshold = df[col_area].quantile(0.90)
    # Filter high intensity
    hi_peaks = df[df[col_area] >= threshold][[col_volume, col_conc_b]].copy()
    # Round to create a "signature" for aggregation
    hi_peaks['sig_vol'] = hi_peaks[col_volume].round(1)
    hi_peaks['sig_conc'] = hi_peaks[col_conc_b].round(1)
    hi_peaks['run_no'] = run_no
    all_high_intensity_peaks.append(hi_peaks)

majority_signatures = pd.DataFrame(columns=['sig_vol', 'sig_conc'])

if all_high_intensity_peaks:
    combined_hi = pd.concat(all_high_intensity_peaks, ignore_index=True)
    # Count occurrences of each signature
    signature_counts = combined_hi.groupby(['sig_vol', 'sig_conc']).size().reset_index(name='count')
    total_runs = len(all_runs_data)
    majority_threshold = total_runs / 2
    majority_signatures = signature_counts[signature_counts['count'] > majority_threshold][['sig_vol', 'sig_conc']]

# Now process each run, filtering by majority signatures
for run_no, df in all_runs_data:
    print(f"Processing run: {run_no}")

    # Re-identify columns for this run
    col_conc_b = 'Conc_B_%'
    col_volume = 'volume_ml'
    col_flow = 'System_flow_ml_min'

```



```

col_area = None
col_uv = 'UV_2_260_mAU' # Defined per feedback, used in logic below

for candidate in ['peak_area_mAU_ml', 'Peak_Area', 'Area', 'Intensity']:
    if candidate in df.columns:
        col_area = candidate
        break

# Verify required columns including System_flow_ml_min
if col_area is None or not all(c in df.columns for c in [col_conc_b,
    col_volume, col_area, col_flow]):
    continue

# 1. Filter for high intensity (top 10% by area) AND UV intensity (top
    10%)
threshold_area = df[col_area].quantile(0.90)
high_intensity_mask = df[col_area] >= threshold_area

# Integrate UV_2_260_mAU into high-intensity filtering logic
if col_uv in df.columns:
    threshold_uv = df[col_uv].quantile(0.90)
    uv_mask = df[col_uv] >= threshold_uv
    # Combine with AND condition as per feedback (secondary metric)
    high_intensity_mask = high_intensity_mask & uv_mask
else:
    # If UV column missing, rely only on Area
    pass

# Apply majority filter if signatures exist
if not majority_signatures.empty:
    # Create signature columns for current run
    df['sig_vol'] = df[col_volume].round(1)
    df['sig_conc'] = df[col_conc_b].round(1)

    # Use merge with indicator to correctly align indices
    merged_check = df[['sig_vol', 'sig_conc']].merge(
        majority_signatures,
        on=['sig_vol', 'sig_conc'],
        how='left',
        indicator=True
    )
    # Keep only rows that matched (both columns found in majority)
    majority_mask = merged_check['_merge'] == 'both'
    high_intensity_mask = high_intensity_mask & majority_mask

    # Clean up temporary columns
    df.drop(columns=['sig_vol', 'sig_conc'], inplace=True)

high_intensity_df = df[high_intensity_mask]

if high_intensity_df.empty:
    # If no peaks left after filtering, skip or return empty
    continue

# 2. Normalize retention volumes
high_intensity_df['normalized_volume'] = high_intensity_df[col_volume] /
    high_intensity_df[col_flow]

# 3. Apply DBSCAN

```

```

X = high_intensity_df[[col_conc_b]].values
clustering = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES, metric='euclidean').
    fit(X)
high_intensity_df['cluster_label'] = clustering.labels_

cluster_counts = high_intensity_df['cluster_label'].value_counts()
if cluster_counts.empty:
    continue

valid_clusters = cluster_counts[cluster_counts.index != -1]

if valid_clusters.empty:
    continue

target_cluster_id = valid_clusters.idxmax()
target_cluster_df = high_intensity_df[high_intensity_df['cluster_label']
    == target_cluster_id]

target_mean = target_cluster_df[col_conc_b].mean()
target_std = target_cluster_df[col_conc_b].std()

window_lower = target_mean - target_std
window_upper = target_mean + target_std

# Logic Flow: Detect (Derivative) -> Fit (Gaussian) -> Calculate SNR ->
    Filter (SNR <= 3) -> Categorize (Target/Critical/Potential)

# Check for unique identifier columns
if 'peak_id' in df.columns:
    id_col = 'peak_id'
elif 'Fraction_unique_ID' in df.columns:
    id_col = 'Fraction_unique_ID'
else:
    id_col = 'generated_peak_id'
    df[id_col] = df.index.astype(str) + '_' + run_no

# Vectorized boolean masking for categorization
mask_target = (df[col_conc_b] >= window_lower) & (df[col_conc_b] <=
    window_upper)

# Critical Anomaly: Outside gradient AND high intensity
mask_outside_gradient = (df[col_conc_b] < GRADIENT_LOW) | (df[col_conc_b]
    > GRADIENT_HIGH)
mask_high_intensity = df[col_area] >= threshold_area
mask_critical = mask_outside_gradient & mask_high_intensity

# Potential Impurity: Within gradient but outside target window
mask_within_gradient = (df[col_conc_b] >= GRADIENT_LOW) & (df[col_conc_b]
    <= GRADIENT_HIGH)
mask_outside_target = ~mask_target
mask_potential = mask_within_gradient & mask_outside_target

# Implement missing classification logic: Dead Time and Inter-Fraction
# These are used to filter non-fractionated peaks or categorize them
    specifically
mask_dead_time = df[col_volume] < VOLUME_DEAD_TIME_THRESHOLD
mask_inter_fraction = (df[col_volume] > VOLUME_DEAD_TIME_THRESHOLD) & (df[
    col_conc_b] < CONC_B_INTER_FRACTION_THRESHOLD)

```

```

# Construct Result DataFrame Directly from Boolean Masks
# We only want rows that are anomalies (Critical or Potential)
# Dead Time and Inter-Fraction are typically not reported as "Anomalies"
  in the target analysis
# unless specified, but the feedback says "Ensure these classifications
  are reflected in the output or used to filter".
# We will filter them out from the anomaly list if they are not Critical/
  Potential,
# or treat them as a separate category if the output schema allowed.
# Given the output schema (Critical/Potential), we will exclude Dead Time/
  Inter-Fraction from the anomaly list
# unless they also meet Critical/Potential criteria (which is unlikely for
  Dead Time).

mask_anomaly = mask_critical | mask_potential

# Filter out Dead Time and Inter-Fraction from the anomaly list if they
  are not already Critical/Potential
# Assuming Dead Time/Inter-Fraction are not "Anomalies" of interest for
  this specific report
mask_excluded = mask_dead_time | mask_inter_fraction
mask_final_anomaly = mask_anomaly & ~mask_excluded

if mask_final_anomaly.any():
    anomaly_df = df[mask_final_anomaly].copy()

    # Assign category
    anomaly_df['anomaly_category'] = 'Potential' # Default
    anomaly_df.loc[mask_critical & mask_final_anomaly, 'anomaly_category']
        = 'Critical'

    # Select required columns and rename for output
    results = anomaly_df[['id_col', 'anomaly_category', col_volume]]
    results.columns = ['anomaly_peak_id', 'anomaly_category', '
        anomaly_retention_volume_ml']

    # Add target stats
    results['run_no'] = run_no
    results['target_peak_conc_b_mean'] = target_mean
    results['target_peak_conc_b_std'] = target_std

    # Reorder columns
    results = results[['run_no', 'target_peak_conc_b_mean', '
        target_peak_conc_b_std', 'anomaly_peak_id', 'anomaly_category', '
        anomaly_retention_volume_ml']]

    # Limit to 20 lines
    if len(results) > MAX_OUTPUT_LINES:
        results = results.head(MAX_OUTPUT_LINES)

    output_filename = f"target_analysis_run_{run_no}.csv"
    output_path = os.path.join(OUTPUT_DIR, output_filename)
    results.to_csv(output_path, index=False)
    print(f"Saved: {output_filename}")
else:
    # No anomalies
    result_df = pd.DataFrame([{'
        'run_no': run_no,
        'target_peak_conc_b_mean': target_mean,

```

```

        'target_peak_conc_b_std': target_std,
        'anomaly_peak_id': None,
        'anomaly_category': None,
        'anomaly_retention_volume_ml': None
    })
    output_filename = f"target_analysis_run_{run_no}.csv"
    output_path = os.path.join(OUTPUT_DIR, output_filename)
    result_df.to_csv(output_path, index=False)
    print(f"Saved: {output_filename}")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks, correlate
import warnings

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_PLOT = '/workspace/purity_correlation.png'
TOLERANCE_FACTOR = 0.5

# Load Data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)

# Ensure numeric types for known columns
cols_to_numeric = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
for col in cols_to_numeric:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Fallback Logic: Verify Variable Names and Calculate Missing Columns
peak_id_col = None
fwhm_col = None

# Check for potential column names for peak_id
potential_peak_ids = ['peak_id', 'Fraction_unique_ID', 'Fraction_number', 'Peak_ID']
for col in potential_peak_ids:
    if col in df.columns:
        peak_id_col = col
        break

# Check for potential column names for fwhm
potential_fwhms = ['fwhm_ml', 'FWHM', 'peak_width', 'width_ml']
for col in potential_fwhms:
    if col in df.columns:
        fwhm_col = col
        break

# Vectorized Peak Detection Fallback
if peak_id_col is None:

```

```

print("Warning: 'peak_id' column not found. Attempting to detect peaks per run.")
df['temp_peak_id'] = -1
for run_val, run_group in df.groupby('run_no'):
    if 'UV_1_280_mAU' not in run_group.columns:
        continue
    peaks, _ = find_peaks(run_group['UV_1_280_mAU'], prominence=0.1)
    if len(peaks) > 0:
        peak_centers = run_group.index[peaks]
        # Vectorized distance calculation for nearest peak assignment
        vol_vals = run_group['volume_ml'].values
        center_vols = run_group.loc[peak_centers, 'volume_ml'].values
        # Calculate distances matrix efficiently
        distances = np.abs(vol_vals[:, None] - center_vols[None, :])
        nearest_peak_idx = np.argmin(distances, axis=1)
        # Assign to original df using explicit index alignment
        df.loc[run_group.index, 'temp_peak_id'] = nearest_peak_idx
    else:
        df.loc[run_group.index, 'temp_peak_id'] = 0
peak_id_col = 'temp_peak_id'
df[peak_id_col] = df[peak_id_col].astype(int)

if fwhm_col is None:
    print("Warning: 'fwhm_ml' column not found. Calculating FWHM dynamically.")
    fwhm_col = 'calculated_fwhm'

# Filter out rows with missing critical data
# Explicit check for UV columns before adding to critical_cols to prevent KeyError
critical_cols = ['run_no', 'volume_ml', peak_id_col]
if 'UV_1_280_mAU' in df.columns:
    critical_cols.append('UV_1_280_mAU')
if 'UV_2_260_mAU' in df.columns:
    critical_cols.append('UV_2_260_mAU')

df_clean = df.dropna(subset=critical_cols)

# Ensure fwhm is numeric if it exists in original data
if fwhm_col in df.columns:
    df_clean[fwhm_col] = pd.to_numeric(df_clean[fwhm_col], errors='coerce')
    df_clean = df_clean.dropna(subset=[fwhm_col])

# Step 1 & 2: Extract signals within FWHM tolerance and handle misalignment
def process_peak_group(group, peak_id_col, fwhm_col, has_original_fwhm):
    """
    Process a single peak group.
    Encapsulated to avoid global variable reliance.
    """
    if len(group) < 5:
        return None

    # Calculate peak center
    idx_max = group[peak_id_col].idxmax()
    center_vol = group.loc[idx_max, 'volume_ml']

    # Get FWHM
    if has_original_fwhm:
        fwhm = group[fwhm_col].iloc[0]
    else:
        # Calculate FWHM dynamically

```

```

max_val = group['UV_1_280_mAU'].max()
half_max = max_val / 2
mask_half = group['UV_1_280_mAU'] >= half_max

if mask_half.sum() < 2:
    # Refined FWHM Fallback: Estimate based on 10% threshold or default
    low_threshold = max_val * 0.1
    mask_low = group['UV_1_280_mAU'] >= low_threshold
    if mask_low.sum() >= 2:
        peak_indices = group[mask_low].index
        vol_start = group.loc[peak_indices.min(), 'volume_ml']
        vol_end = group.loc[peak_indices.max(), 'volume_ml']
        fwhm = vol_end - vol_start
    else:
        fwhm = 1.0 # Default resolution
else:
    peak_indices = group[mask_half].index
    # Ensure peak_indices is not empty before accessing min/max
    if len(peak_indices) > 0:
        vol_start = group.loc[peak_indices.min(), 'volume_ml']
        vol_end = group.loc[peak_indices.max(), 'volume_ml']
        fwhm = vol_end - vol_start
    else:
        fwhm = 1.0

# Define window
lower_bound = center_vol - (TOLERANCE_FACTOR * fwhm)
upper_bound = center_vol + (TOLERANCE_FACTOR * fwhm)

mask = (group['volume_ml'] >= lower_bound) & (group['volume_ml'] <=
upper_bound)
peak_data = group[mask]

if len(peak_data) < 3:
    return None

u280 = peak_data['UV_1_280_mAU'].values
u260 = peak_data['UV_2_260_mAU'].values

# Normalize for correlation
if np.std(u280) == 0 or np.std(u260) == 0:
    u260_shifted = u260
else:
    u280_norm = (u280 - np.mean(u280)) / (np.std(u280) * len(u280))
    u260_norm = (u260 - np.mean(u260)) / (np.std(u260) * len(u260))

corr = correlate(u260_norm, u280_norm, mode='full')
lag = np.argmax(corr) - (len(u280) - 1)

if abs(lag) > 1:
    if lag > 0:
        u260_shifted = np.roll(u260, lag)
        u260_shifted[:lag] = 0
    else:
        u260_shifted = np.roll(u260, lag)
        u260_shifted[lag:] = 0
else:
    u260_shifted = u260

```

```

volumes = peak_data['volume_ml'].values
auc_280 = np.trapz(u280, volumes)
auc_260 = np.trapz(u260_shifted, volumes)

ratio = auc_260 / auc_280 if auc_280 != 0 else np.nan

# Calculate SNR for Critical Anomaly logic
# Estimate noise as std of baseline (outside peak window or low signal region)
baseline_mask = group['UV_1_280_mAU'] < (group['UV_1_280_mAU'].max() * 0.1)
if baseline_mask.sum() > 10:
    noise_level = group.loc[baseline_mask, 'UV_1_280_mAU'].std()
else:
    noise_level = np.std(u280) * 0.1 # Fallback

snr = auc_280 / noise_level if noise_level > 0 else 0

return {
    'run_no': group['run_no'].iloc[0],
    'peak_id': group[peak_id_col].iloc[0],
    'center_vol': center_vol,
    'auc_280': auc_280,
    'auc_260': auc_260,
    'ratio_260_280': ratio,
    'snr': snr,
    'fwhm': fwhm,
    'mask': mask # Return mask to avoid re-calculation
}

# Group by run and peak
df_clean[peak_id_col] = df_clean[peak_id_col].astype(str)
groups = df_clean.groupby(['run_no', peak_id_col])

results_list = []
cached_peak_subsets = {}

print("Processing peaks...")
# Merged Processing and Caching Loops
has_original_fwhm = fwhm_col in df.columns
for name, group in groups:
    res = process_peak_group(group, peak_id_col, fwhm_col, has_original_fwhm)
    if res:
        results_list.append(res)
        # Cache the subset directly using the mask returned
        cache_key = (group['run_no'].iloc[0], group[peak_id_col].iloc[0])
        cached_peak_subsets[cache_key] = group[res['mask']]

# Create DataFrame for summary
summary_df = pd.DataFrame(results_list)

# Step 4: Visualize Correlation
print("Generating visualization...")

# Vectorize the aggregation of points
all_points = []
for cache_key, subset in cached_peak_subsets.items():
    run_no = subset['run_no'].iloc[0]
    # Column Existence Checks in Plotting
    if 'UV_1_280_mAU' in subset.columns and 'UV_2_260_mAU' in subset.columns:
        points_df = subset[['UV_1_280_mAU', 'UV_2_260_mAU']].copy()

```

```

        points_df['run_no'] = run_no
        points_df.columns = ['x', 'y', 'run_no']
        all_points.append(points_df)
    else:
        print(f"Warning: Missing UV columns in subset for {cache_key}. Skipping plot.")

if all_points:
    plot_df = pd.concat(all_points, ignore_index=True)
else:
    plot_df = pd.DataFrame(columns=['x', 'y', 'run_no'])

if len(plot_df) == 0:
    print("No data points found for plotting.")
    plt.figure()
    plt.title("No Data")
    plt.savefig(OUTPUT_PLOT)
else:
    plt.figure(figsize=(10, 8))

    run_counts = plot_df.groupby('run_no').size()
    max_points_per_run = run_counts.max()

    if max_points_per_run <= 100:
        # <= 100 points: Aggregate by run, calculate mean and 95% CI, plot with error bars
        agg = plot_df.groupby('run_no').agg(
            mean_x=('x', 'mean'),
            mean_y=('y', 'mean'),
            std_x=('x', 'std'),
            std_y=('y', 'std'),
            count=('x', 'count')
        ).reset_index()

        # Calculate 95% CI
        agg['ci_x'] = 1.96 * agg['std_x'] / np.sqrt(agg['count'])
        agg['ci_y'] = 1.96 * agg['std_y'] / np.sqrt(agg['count'])

        colors = plt.cm.tab10(np.linspace(0, 1, len(agg)))
        for i, row in agg.iterrows():
            plt.errorbar(row['mean_x'], row['mean_y'],
                        xerr=row['ci_x'], yerr=row['ci_y'],
                        fmt='o', color=colors[i], label=f"Run_{row['run_no']}",
                        capsize=3, alpha=0.8)

        plt.legend(title="RunNo", bbox_to_anchor=(1.05, 1), loc='upper_left')
        plt.title("Protein vs Nucleic Acid Signal Correlation by Run")
    else:
        # > 100 points: Use scatter with alpha to show density and color by run
        # This satisfies "Color points by run_no" while handling density via transparency
        scatter = plt.scatter(plot_df['x'], plot_df['y'],
                             c=plot_df['run_no'], cmap='tab10', alpha=0.3, s=20)
        plt.colorbar(scatter, label="RunNo")
        plt.title("Protein vs Nucleic Acid Signal Correlation by Run")
        plt.legend(title="RunNo", bbox_to_anchor=(1.05, 1), loc='upper_left')

plt.xlabel("UV_1_280_mAU (Protein)")
plt.ylabel("UV_2_260_mAU (Nucleic Acid)")

```



```

plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig(OUTPUT_PLOT, dpi=300)
print(f"Plot saved to {OUTPUT_PLOT}")

# Step 5 & 6: Identify contaminants and generate summary
if not summary_df.empty:
    # Explicit SNR calculation used in logic
    # Filter Critical Anomalies based on SNR threshold (e.g., SNR <= 3)
    summary_df['is_low_snr'] = summary_df['snr'] <= 3

    # Contaminant identification relies exclusively on ratio_260_280
    # Data-driven threshold: 95th percentile
    threshold = summary_df['ratio_260_280'].quantile(0.95)
    summary_df['is_contaminant'] = summary_df['ratio_260_280'] > threshold

    run_summary = summary_df.groupby('run_no').agg(
        total_peaks=('peak_id', 'count'),
        avg_ratio=('ratio_260_280', 'mean'),
        max_ratio=('ratio_260_280', 'max'),
        contaminant_peaks=('is_contaminant', 'sum'),
        low_snr_peaks=('is_low_snr', 'sum')
    ).reset_index()

    print("\n--- Purity Summary per Run ---")
    print(run_summary.to_string(index=False))
    print(f"Threshold used for contaminants (Ratio): {threshold:.4f}")
    print(f"SNR Threshold for anomalies: 3.0")
else:
    print("No peaks processed. Summary empty.")

```

Listing 3: Code_3